

Prim's Algorithm – Complete Documentation

1. What is Prim's Algorithm?

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a weighted connected graph. The algorithm starts from one vertex and gradually grows the tree by repeatedly choosing the smallest edge that connects the already-visited vertices to an unvisited vertex.

Goal

Find a tree that:

- ✓ Contains all vertices.
- ✓ Has exactly $V - 1$ edges.
- ✓ Has no cycles.
- ✓ Has minimum total weight.

Original Graph

Consider the graph described below:

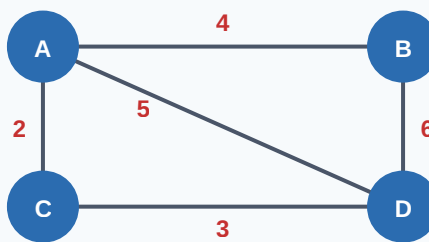


Figure 1: Input Weighted Connected Graph

Edges Summary

Edge	Weight
AC	2
CD	3
AB	4
AD	5
BD	6

Notice: There is NO edge between B and C.

Idea Behind Prim's Algorithm

Suppose some vertices are already inside the tree.

```
Visited Vertices
      |
      | minimum edge
      ↓
Unvisited Vertex
```

At every step: Choose the smallest edge that connects the current tree to an unvisited vertex.

Why Does Prim's Algorithm Never Form a Cycle?

A cycle occurs only when both ends of an edge are already inside the tree. Prim's algorithm always chooses:

Visited Vertex —————> **Unvisited Vertex**

Since one endpoint is outside the tree, no cycle can be created. Therefore, **cycle detection is not required**.

Step-by-Step Dry Run

Step 0

Start from vertex **A**.

Visited: A | **Tree:** A | **Cost:** 0

Available Edges from A: AB = 4, AC = 2, AD = 5

Choose the smallest edge: **AC = 2**

Step 1

Tree becomes: A — 2 — C

Visited: A, C | **Current Cost:** 2

Available Edges Now:

- From A: AB = 4, AD = 5
- From C: CD = 3

Total available edges: AB = 4, AD = 5, CD = 3

Choose the smallest: **CD = 3**

Step 2

Tree becomes: A — 2 — C — 3 — D

Visited: A, C, D | **Cost:** 2 + 3 = 5

Available Edges Now:

- From A: AB = 4, AD = 5
- From D: BD = 6

Total available edges: AB = 4, AD = 5, BD = 6

Choose the smallest: **AB = 4**

Step 3 (Final MST)

Tree becomes: B — 4 — A — 2 — C — 3 — D

Visited: A, B, C, D | **Cost:** $2 + 3 + 4 = 9$

Since there are $V = 4$ vertices, the required number of edges is $V - 1 = 3$. We already have 3 edges, so the algorithm stops.

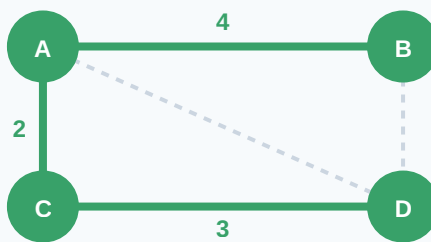
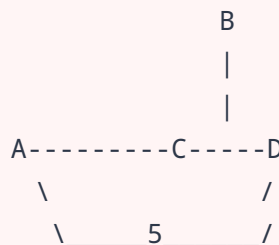


Figure 2: Final Minimum Spanning Tree (Total Cost = 9)

Why Was AD = 5 Not Chosen?

Current tree: B — A — C — D

Suppose we add AD:



A cycle would be formed: $A \rightarrow C \rightarrow D \rightarrow A$. Therefore, AD is ignored.

General Algorithm

- **Step 1:** Choose any starting vertex.
- **Step 2:** Mark it as visited.
- **Step 3:** Among all edges that connect **Visited Vertex** → **Unvisited Vertex**, choose the minimum weight edge.
- **Step 4:** Add that edge to the MST.
- **Step 5:** Mark the new vertex as visited.
- **Step 6:** Repeat until **Number of edges** = $V - 1$.

Pseudocode

```
Prim(G)
  Start from any vertex.
  while all vertices are not visited
    choose the minimum edge connecting visited and unvisited vertices
    add that edge to MST
    mark the new vertex as visited
  return MST
```

Efficient Implementation

We use:

1. Adjacency List

Stores graph efficiently. Example representation for our graph:

```
0 : (1,4) (2,2) (3,5)
1 : (0,4) (3,6)
2 : (0,2) (3,3)
3 : (0,5) (1,6) (2,3)
```

2. Min Heap (Priority Queue)

Always gives the smallest edge first. It stores elements as `(weight, node)`.

Example heap state during execution:

```
(2,C)
(4,B)
(5,D)
```

Top element: (2,C)

```
int primMST(int V, vector<vector<int>> adj[])
{
    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;

    vector<bool> visited(V,false);

    pq.push({0,0});

    int sum = 0;

    while(!pq.empty())
    {
        int weight = pq.top().first;
        int node = pq.top().second;

        pq.pop();

        if(visited[node])
            continue;

        visited[node] = true;

        sum += weight;

        for(auto neighbour : adj[node])
        {
            int nextNode = neighbour[0];
            int nextWeight = neighbour[1];

            if(!visited[nextNode])
            {
                pq.push({nextWeight,nextNode});
            }
        }
    }

    return sum;
}
```

Code Explanation

- **Create Min Heap:** `priority_queue<...> pq;` stores `(weight, node)` where the smallest weight comes first.
- **Start From Node 0:** `pq.push({0, 0});` uses weight 0 because no edge has been chosen yet.
- **Get Minimum Edge:** Extracts the smallest edge via `pq.top()` and `pq.pop()`.
- **Skip Already Visited Nodes:** `if(visited[node]) continue;` prevents loops and revisiting.
- **Add Weight To MST:** `sum += weight;` inserts the current edge into the MST cost.
- **Explore Neighbors:** `for(auto neighbour : adj[node])` traverses all adjacent vertices.
- **Insert Unvisited Vertices Into Heap:** `pq.push({nextWeight, nextNode});` automatically keeps the minimum edge at the top.

Complexity Analysis

- **Time Complexity:** $O(E \log V)$ where V is the number of vertices and E is the number of edges.
- **Space Complexity:** $O(V + E)$ due to the Adjacency List and Priority Queue.

Intuition

Imagine building a road network. Initially, you are at city **A**. You always construct the cheapest road that reaches a new city:

Visited Cities — cheapest road —> New City

Gradually expanding:

$A \rightarrow A-C \rightarrow A-C-D \rightarrow B-A-C-D$

until every city becomes connected.

One-Line Definition: Prim's algorithm constructs the Minimum Spanning Tree by repeatedly selecting the smallest edge that connects a visited vertex to an unvisited vertex.